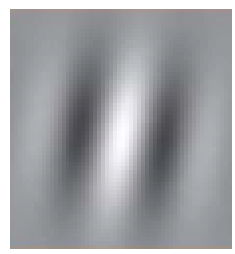
Model fitting tutorial

PSYCHOPHYSICS -- Fall 2020 -- Created by Shannon Locke

Overview

- **Matlab:**
  - Module 1A: Fitting a psychometric function using maximum likelihood estimation
  - Module 1B: Comparing model fits
- **R:**
  - Module 1C: Fitting a psychometric function using (ML and) MCMC in R
  - Module 2: GLMMs
  - Module 3: A hierarchical model in RStan

Module 1A: Fitting a psychometric function using maximum likelihood estimation - Shannon

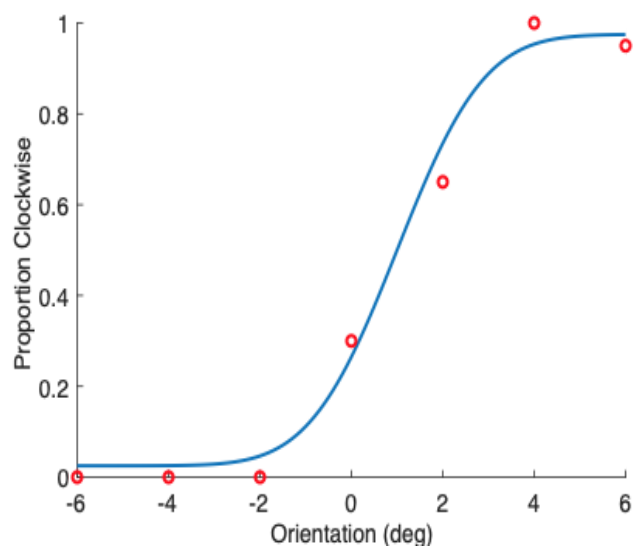


Goal: Demonstrate different methods of optimization (fminsearch, fmincon, BADS, and grid brute-force).

Example: Fitting the psychometric function of a biased observer in an orientation discrimination task measured with the method of constant stimuli.

Functions:

- *Module_1A.m*: The main function that simulates the observer and fits the models.
- *model_nLL.m*: computes the negative log-likelihood for a specific parameter combination (mu,sigma). Note that we are keeping the lapse rate fixed in this experiment.
- *model_nLL_bySim.m*: same as model_nLL.m but computes the probability of a clockwise judgement for each trial by simulation. This example is silly because we know the form of the psychometric function, but it does illustrate how to fit by simulation.



Key equations:

- Bernoulli distribution: $\mathrm{Bern}(x|\mu) = \mu^x(1-\mu)^{1-x}$

Step-by-step:

1. We simulate the responses of an observer with mu=1 and sigma=1.5.
2. Use Matlab's fminsearch function to find the minimum negative log-likelihood. We only provide a starting location for the gradient descent search. It is usually good practice to run fminsearch with multiple start-points to check for convergence. Fminsearch uses the Nelder-Mead approach.
3. We repeat using Matlab's fmincon function, which allows us to also specify lower and upper bounds on the parameter estimates. It's good to use fmincon if you have bounded parameters (e.g., a slope parameter that can't be <0). Fmincon algorithm uses a numerical approximation of the gradient.
4. Fit by Bayesian Adaptive Direct Search (BADS), a package created by NYU's own Luigi Acerbi. It's strengths are 1) it's easy to use, 2) it is fast, and 3) it is very good at fitting models that require simulation. In addition to starting location and lower/upper bounds, you can also specify plausible lower/upper bounds to help improve the fit. BADS uses mesh adaptive direct search with Bayesian optimisation (see resources below for details).
5. Fitting with BADS but with simulating an observer to estimate the probability of a clockwise response per trial (i.e., a noisy objective function). It's important to tell bads that the negative log-likelihood values are stochastic. Notice we used the same method for simulating our fake observer responses and estimating the probability of a clockwise judgement.
6. Brute-force grid method for maximum likelihood estimation. Here we construct a parameter grid of plausible combinations of the two free parameters, and calculate the negative log-likelihood at each point in the grid. To find the maximum likelihood estimate we find the combination that gave the LOWEST negative log-likelihood value. This method is ok up to about a max of 4 parameters.

Questions:

- For experiments with a method of constant stimuli design, how would you improve the efficiency of model_nLL?
- If you were stuck using fminsearch and had a parameter that could not be <0 (e.g., slope), how could you implement this in the model_nLL script?
- How could you improve your estimate of the maximum likelihood when using the brute-force grid method?
- Can you think of scenarios where it makes sense to fit using the brute-force grid method?
- How does the estimated negative log-likelihoods compare to each other? Which method/methods performed best?
- How fast are each of the methods? Which method/methods performed best? What model fitting options affect the time to run the code?
- How would you adjust the code to fit a lapse rate?

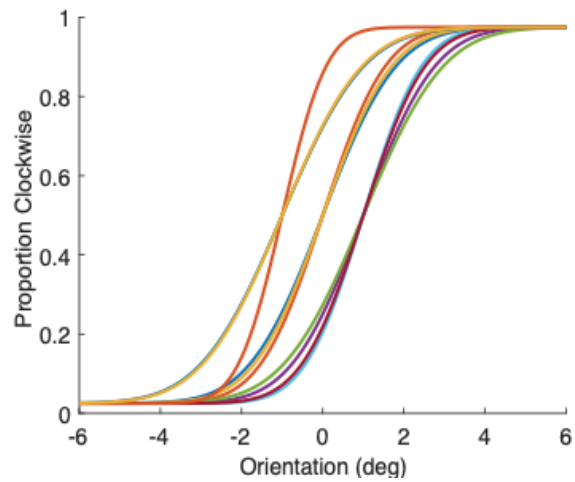
Resources:

- BADS:
  - To download and read the user guide: https://github.com/lacerbi/bads.
  - Cite: *Acerbi, L. & Ma, W. J. (2017). Practical Bayesian Optimization for Model Fitting with Bayesian Adaptive Direct Search. In Advances in Neural Information Processing Systems 30, pages 1834-1844.*

Module 1B: Comparing model fits - Shannon

Goal: Demonstrate different model comparison methods. Some methods use the maximum log likelihood (boot-strapping, AIC, BIC, nested model comparison) and others are "full" Bayesian (Bayes factor, exceedance probabilities, model partitioning).

Example: Ten participants perform the task described in part A. Three (30%) are unbiased and seven (70%) are biased. Some of the biased observers are biased towards reporting "clockwise" (40%) and others "counter-clockwise" (30%) . For simplicity, we have simulated the biased observers to all have a bias magnitude of 1 deg. We perform the model comparison to understand if the mu parameter is necessary. We considered 4 models: 1) unbiased, mu=0, 2) biased CW, mu=1, 2) biased CCW, mu=-1, 3) any bias, mu is a free parameter.



Functions:

- *Module_1B.m*: The main function that demonstrates model comparison methods.
- *getModelEvidence.m*: This is a general function for computing the model evidence from any brute-force grid fit. You will not receive this code until after the homeworks have been submitted,

Key equations:

- Non-Bayesian:
  - AIC = $-2\log p(y\,|\,\hat{\theta}, M) + 2K$, and the corrected version is AICc = $\text{AIC} + \frac{2K(K+1)}{N-K-1}$
  - BIC = $-2\log p(y\,|\,\hat{\theta}, M) + K\log N$ with $\text{Bayes factor} \approx \exp\left(-\frac{1}{2}(BIC_1 - BIC_2)\right)$
  - Nested model comparison: $(2\log \frac{p(y\,|\,\hat{\theta}_{\text{complex}}, M_{\text{complex}})}{p(y\,|\,\hat{\theta}_{\text{simple}}, M_{\text{simple}})} \sim \chi^2(K)$

- Bayesian:
  - Model Evidence: $p(y|M_1) = \int p(y|\theta, M_1)p(\theta|M_1)d\theta$
  - Bayes factor = $\frac{p(y|M_1)}{p(y|M_2)}$
  - Exceedance Probability (EP): $EP_i = P\left(r_i > r_j \mid y\right)$ where $j \neq i$. where *r* is the population frequency profile.
  - Bayesian Omnibus Risk (BOR): i.e., BOR = P(r_i = r_j) for all i,j
  - Protected Exceedance Probability (PEP) = (1 - BOR) * EP + BOR/N_models

Step-by-step:

7. Bootstrapping (parametric and non-parametric): bootstrapping allows you to get confidence intervals on your parameter estimates for 1 or more models. It is performed by sampling with replacement either from the original dataset (non-parametric) or the best-fitting function (parametric) to get N new datasets. You then fit these new datasets and look at the distribution of estimated parameters. This will allow you to see how well the model is fitting the data as well as understand if certain parameters are necessary (e.g., significantly different from 0 or 1) or differ between experimental conditions of interest.
8. AIC and AICc: A simple non-Bayesian way of calculating model fit with the maximum log-likelihood values. Remember that only the relative AIC scores are of interest (i.e., comparing for different models).
9. BIC and approx. Bayes factor: BIC is another method of comparing models with log-likelihood values. It penalises model complexity differently from AIC. BIC scores can also be used to calculate approximate Bayes factor values.
10. Bayes factor: try filling this in yourself
11. Protected Exceedance Probabilities and model partitioning: Exceedance probability analyses consider that participants may differ not only in terms of the best-fitting parameter values but also in terms the underlying psychometric model used to perform the task (e.g., 30\% of people use model A, 50\% model B, and 20\% model C). It also calculates the Bayesian Omnibus Risk (BOR), the posterior probability all models are all equally frequent in the population. Protected exceedance probabilities are a weighting of the prior that all models are equally likely (i.e., flat prior) and the exceedance probabilities (i.e., how well each model fits each observer), with the weights determined by the BOR. Model partitioning considers that models may be grouped in interesting ways (e.g., linear/non-linear, non-Bayesian/Bayesian, etc.). It groups together model evidence by type to answer questions at the model group level (e.g., are linear models better fits than non-linear models?).

Questions:

- What happens to the the AICc values and Protected Exceedance Probabilities when all observers are unbiased?
- What happens when observers have biases or various magnitudes?
- How do the approximate Bayes factor, calculated from the BIC scores, compare to the Bayes factor calculated from the model evidence?

Resources:

- AIC:
  - Cite: *Akaike, H. (1974). A New Look at the Statistical Model Identification. IEEE Transactions on Automatic Control, 19(6):716–723.*
  - For AICc, also cite: *Cavanaugh, J. E. (1997). Unifying the derivations for the Akaike and corrected Akaike information criteria.Statistics & Probability Letters, 33(2):201–208.*
- Protected Exceedance Probabilities:
  - Cite: *Stephan, K. E., Penny, W. D., Daunizeau, J., Moran, R. J., and Friston, K. J. (2009). Bayesian model selection for group studies.Neuroimage, 46(4):1004–1017.*
  - Cite: *Rigoux, L., Stephan, K. E., Friston, K. J., and Daunizeau, J. (2014). Bayesian model selection for group studies—Revisited.NeuroImage, 84:971–985.*
- SPM12:
  - To download and read the user guide: https://www.fil.ion.ucl.ac.uk/spm/
  - Cite in text: *SPM12 Toolbox (Wellcome Trust Centre for Neuroimaging, London, UK)*
  - Doesn't do model partitioning!
- VBA:
  - To download and read the user guide: https://mbb-team.github.io/VBA-toolbox/
  - Cite: *Daunizeau, J., Adam, V., and Rigoux, L. (2014). VBA: A probabilistic treatment of nonlinear models for neurobiological and behavioural data.PLoS Computational Biology, 10(1):e1003441.*
  - Computing protected exceedance probabilities from exceedance probabilities is left up to you!

Module 1C: Fitting a psychometric function using (ML and) MCMC in R - Steph

See Steph's code for details.

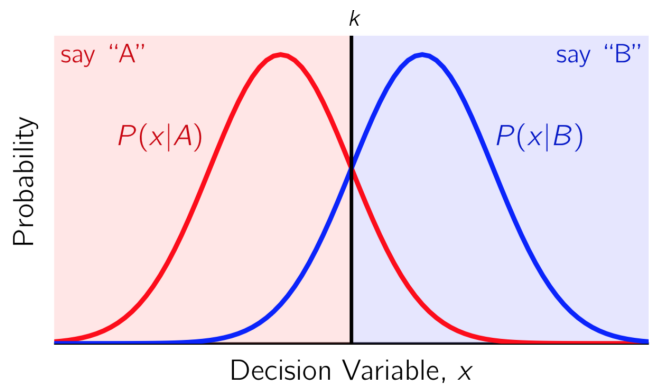
Module 2: GLMMs - Steph

See Steph's code for details.

Module 3: A hierarchical Bayesian model in RStan - Shannon

Goal: Demonstrate how to fit basic and hierarchical d-prime models in RStan. Also, to understand the advantages and disadvantages of using the hierarchical method.

Example: Ten participants perform a simplified version of the orientation discrimination task described in Module 1. This time, they only see one of two stimuli on every trial (either counterclockwise or clockwise). Prior to the main experiment, each observer completed a staircase calibration procedure to equate stimulus difficulty to *d'=1*. There is also no reason to suspect an observer will be biased in this task.



Functions:

- *Module_3.m*: A Matlab function to simulate the observers and compare the fits computed via different methods.
- *Module_3.R*: A R script for fitting the different RStan models and exporting the results.
- *model_SDT_basic.stan*: A RStan file of the basic model that computes *d'* and *k* for a single observer.
- *model_SDT_hierarchical.stan*: A RStan file of the hierarchical Bayesian model that computes *d'* and *k* for multiple observers. It assumes that at the population level, *d'=1* and *k=0*.

Questions:

- What are the hyperparameters in this example? What are the hyperpriors?
- What does the MCMC sampling method give you that the standard formulas for d' and k do not?
- How do the SD estimates for d' and k compare to the values we used for simulation? Why are they not the same?
- Are 50 repeats (i.e., 100 trials) good enough for a good fit of the SDT model?
- How would you apply a fixed lapse rate to this SDT model? What about a variable lapse rate?
- Is flat prior suitable for the criterion? What other prior might you consider?
- How do the estimates of the population-level parameters change as you add more stimulus repeats in the experiment?
- What happens to the subject-level and population-level estimates when you add more participants?
- How would you include another layer in the hierarchical model if you collected 5-sessions of data per subject, and didn't want to assume that *d'* and *k* are the same for all sessions?

Resources:

- RStan:
  - Cite: *Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., and Riddell, A. (2017). Stan: A probabilistic programming language. Journal of Statistical Software, 76(1):1–32.*
  - *See also the shinystan package for visualising the model fits*